amazon

# Amazon E-Commerce SQL Analysis

A 14-Question Business Case Study Across Sellers, Orders, Products & Customer Reviews

Prepared by Kunal Malviya | SQL Case Study | 11-Table Relational Dataset

# Index


01 Orders Fulfilled by Seller State

02 Cumulative Revenue by Product Category Over Time

03 Most-Used Payment Method & Average Order Value

04 Highest-Spending Customer

05 Average Review Score by Product Category

06 Orders per Customer by State

07 Sellers With Zero Fulfilled Orders

08 Top 5 Product Categories by Revenue

09 Median Delivery Time

10 Products Never Ordered

11 Sellers Above the Average Order Count

12 Highest-Rated States by Average Review Score

13 Customers Who Never Left a Review

14 Month With the Highest Order Volume

# 01 Orders Fulfilled by Seller State

```sql
CREATE INDEX abc
ON orders(order_status);

SELECT s.seller_state,
COUNT(DISTINCT o.order_id) AS order_count
FROM sellers s
JOIN order_items oi
ON s.seller_id = oi.seller_id
JOIN orders o
ON oi.order_id = o.order_id
WHERE o.order_status = "delivered"
GROUP BY s.seller_state
ORDER BY order_count DESC;
```

**Output**

| seller_state | order_count |
|---|---|
| SP | 68641 |
| MG | 7735 |
| PR | 7512 |
| RJ | 4227 |
| SC | 3603 |
| RS | 1962 |
| DF | 808 |
| BA | 550 |
| GO | 451 |
| PE | 403 |
| MA | 389 |
| ES | 310 |
| MT | 136 |
| CE | 87 |
| RN | 51 |

SP, MG, and PR lead in order volume, while states like SE, PI, and RO have too few orders to justify investing in infrastructure.

02

# Cumulative Revenue by Product Category Over Time

amazon

```sql
WITH a AS (SELECT p.product_category_name,
DATE(o.order_purchase_timestamp) AS order_date,
ROUND(SUM(oi.price),2) AS revenue
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
JOIN orders o
ON oi.order_id = o.order_id
GROUP BY p.product_category_name, order_date)

SELECT *,
ROUND(SUM(revenue) OVER(PARTITION BY product_category_name
ORDER BY order_date),2) AS cumulative_revenue
FROM a;
```

**Output**

| product_category_name | order_date | revenue | cumulative_revenue |
|---|---|---|---|
| utilidades_domesticas | 2016-10-04 | 289.7 | 289.7 |
| utilidades_domesticas | 2016-10-06 | 399.8 | 689.5 |
| utilidades_domesticas | 2016-10-07 | 329.7 | 1019.2 |
| utilidades_domesticas | 2016-10-08 | 189.89 | 1209.09 |
| utilidades_domesticas | 2016-10-09 | 44.99 | 1254.08 |
| utilidades_domesticas | 2016-10-10 | 32.99 | 1287.07 |
| utilidades_domesticas | 2017-01-12 | 19.9 | 1306.97 |
| utilidades_domesticas | 2017-01-13 | 498.99 | 1805.96 |
| utilidades_domesticas | 2017-01-18 | 173.85 | 1979.81 |
| utilidades_domesticas | 2017-01-19 | 509.8 | 2489.61 |
| utilidades_domesticas | 2017-01-22 | 14.5 | 2504.11 |
| utilidades_domesticas | 2017-01-23 | 347.7 | 2851.81 |
| utilidades_domesticas | 2017-01-24 | 284.79 | 3136.6 |
| utilidades_domesticas | 2017-01-25 | 80.4 | 3217 |
| utilidades_domesticas | 2017-01-26 | 374.15 | 3591.15 |
| utilidades_domesticas | 2017-01-28 | 60 | 3651.15 |
| utilidades_domesticas | 2017-01-29 | 252.6 | 3903.75 |
| utilidades_domesticas | 2017-01-30 | 166.4 | 4070.15 |

This table makes it easy to identify which category reached its target early in the month.

# 03 Most-Used Payment Method & Average Order Value

amazon

```sql
SELECT payment_type,
COUNT(DISTINCT order_id) AS order_count,
ROUND(AVG(payment_value),2) AS average_order_value
FROM order_payments
GROUP BY payment_type
ORDER BY order_count DESC;
```

**Output**

| payment_type | order_count | average_order_value |
|---|---|---|
| credit_card | 76505 | 163.32 |
| boleto | 19784 | 145.03 |
| voucher | 3866 | 65.7 |
| debit_card | 1528 | 142.57 |
| not_defined | 3 | 0 |

**The results table shows that the credit card is the most commonly used payment method, with the highest average order value as well.**

# 04 Highest-Spending Customer

amazon

```sql
SELECT o.customer_id,
SUM(op.payment_value) AS money_spent
FROM orders o
JOIN order_payments op
ON o.order_id = op.order_id
GROUP BY o.customer_id
ORDER BY money_spent DESC
LIMIT 1;
```

**Output**

| customer_id | money_spent |
|---|---|
| 1617b1357756262bfa56ab541c47bc16 | 13664.08 |

Without using the LIMIT clause, we can also view the second- and third-highest spending customers.

# 05 Average Review Score by Product Category

amazon

```
SELECT p.product_Category_name,
ROUND(AVG(r.review_score),2) AS avg_review
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
JOIN order_reviews r
ON oi.order_id = r.order_id
GROUP BY p.product_category_name
ORDER BY avg_review DESC;
```

**Output**

| product_Category_name | avg_review |
|---|---|
| cds_dvds_musicais | 4.64 |
| fashion_roupa_infanto_juvenil | 4.50 |
| livros_interesse_geral | 4.45 |
| construcao_ferramentas_ferramentas | 4.44 |
| flores | 4.42 |
| livros_importados | 4.40 |
| livros_tecnicos | 4.37 |
| alimentos_bebidas | 4.32 |
| malas_acessorios | 4.32 |
| portateis_casa_forno_e_cafe | 4.30 |
| fashion_esporte | 4.26 |
| fashion_calcados | 4.23 |
| alimentos | 4.22 |
| musica | 4.21 |
| cine_foto | 4.21 |
| pet_shop | 4.19 |

**There are many product categories at the bottom that need intervention because of their very low average review scores.**

# 06 Orders per Customer by State

```sql
SELECT c.customer_unique_id, c.customer_state,
COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id, c.customer_state
ORDER BY order_count DESC;
```

**Output**

| customer_unique_id | customer_state | order_count |
|---|---|---|
| 8d50f5eadf50201ccdcedfb9e2ac8455 | SP | 17 |
| 3e43e6105506432c953e165fb2acf44c | SP | 9 |
| 1b6c7548a2a1f9037c1fd3ddfed95f33 | MG | 7 |
| 6469f99c1f9dfae7733b25662e7f1782 | SP | 7 |
| ca77025e7201e3b30c44b472ff346268 | PE | 7 |
| 12f5d6e1cbf93dafd9dcc19095df0b3d | PR | 6 |
| f0e310a6839dce9de1638e0fe5ab282a | ES | 6 |
| de34b1611759416 1a6a89c50b289d35a | SP | 6 |
| dc813062e0fc23409cd255f7f53c7074 | PE | 6 |
| 63cfc61cee11cbe306bff5857d00bfe4 | RJ | 6 |
| 47c1a3033b8b77b3ab6e109eb4d5fdf3 | SP | 6 |
| 4e65032f1f574189fb793bac5a867bbc | SP | 5 |
| 56c8638e7c058b98aae6d74d2dd6ea23 | MG | 5 |
| 74cb1ad7e6d5674325c1f99b5ea30d82 | SP | 5 |
| 5e8f38a9a1c023f3db718edcf926a2db | BA | 5 |

Now they can target low-frequency states with acquisition offers and reward high-frequency states with loyalty perks.

amazon

# 07 Sellers With Zero Fulfilled Orders

```sql
SELECT s.seller_id
FROM sellers s
LEFT JOIN order_items oi
ON s.seller_id = oi.seller_id
LEFT JOIN orders o
ON oi.order_id = o.order_id
WHERE o.order_status = "delivered"
GROUP BY s.seller_id
HAVING COUNT(DISTINCT o.order_id) = 0;
```

**Output**

| | seller_id |
|---|---|

There are no sellers in our data who registered on the platform but have never fulfilled a single order.

amazon

# 08 Top 5 Product Categories by Revenue

```sql
WITH a AS (SELECT p.product_category_name,
ROUND(SUM(oi.price),2) AS revenue
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.product_category_name),

b AS (SELECT *,
DENSE_RANK() over (ORDER BY revenue DESC) AS rnk
FROM a)

SELECT * FROM b
WHERE rnk <= 5;
```

**Output**

| product_category_name | revenue | rnk |
|---|---|---|
| beleza_saude | 1258681.34 | 1 |
| relogios_presentes | 1205005.68 | 2 |
| cama_mesa_banho | 1036988.68 | 3 |
| esporte_lazer | 988048.97 | 4 |
| informatica_acessorios | 911954.32 | 5 |

**These are the top 5 categories by revenue and are well-suited for allocating a promotional budget whenever a sale is planned.**

# 09 Median Delivery Time

amazon

```sql
WITH a AS (SELECT DATEDIFF(order_delivered_customer_date, order_purchase_timestamp)
AS days
FROM orders
WHERE order_delivered_customer_date IS NOT NULL),

b AS (SELECT *,
ROW_NUMBER() OVER(ORDER BY days) AS pos,
COUNT(days) OVER() as n
FROM a)

SELECT AVG(days) AS median
FROM b
WHERE pos IN (CEIL(n/2), CEIL((n+1)/2));
```

**Output**

| | median |
|---|---|
| ▶ | 10.0000 |

The median delivery time (in days) between order placement and actual delivery is 10 days.

# 10 Products Never Ordered

amazon

```sql
SELECT p.product_id
FROM products p
LEFT JOIN order_items oi
ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

**Output**

| | product_id |
|---|---|

There are no products that have never been ordered.

# 11 Sellers Above the Average Order Count

amazon

```sql
WITH a AS (SELECT s.seller_id,
COUNT(DISTINCT CASE
        WHEN o.order_status = "delivered"
        THEN o.order_id
    END) AS order_count
FROM sellers s
LEFT JOIN order_items oi
ON s.seller_id = oi.seller_id
LEFT JOIN orders o
ON oi.order_id = o.order_id
GROUP BY s.seller_id)

SELECT seller_id, order_count
FROM a
WHERE order_count > (SELECT AVG(order_count) FROM a);
```

**Output**

| seller_id | order_count |
|---|---|
| 001cca7ae9ae17fb1caed9dfb1094831 | 195 |
| 002100f778ceb8431b7a1020ff7ab48f | 50 |
| 004c9cd9d87a3c30c522c48c4fc07416 | 156 |
| 00ee68308b45bc5e2660cd833c3f81cc | 135 |
| 00fc707aaaad2d31347cf883cd2dfe10 | 103 |
| 0176f73cc1195f367f7b32db1e5b3aa8 | 37 |
| 01cf7e3d21494c41fb86034f2e714fa1 | 59 |
| 01fdefa7697d26ad920e9e0346d4bd1b | 128 |
| 0241d4d5d36f10f80c644447315af0bd | 238 |
| 02f623a8eb246f3c5f7c2f96462654e6 | 44 |
| 04308b1ee57b6625f47df1d56f00eedf | 92 |
| 0432ead42b6c8a0bdf68154add917fdf | 82 |
| 048c2757535328e0d7dac690ad3c0aae | 50 |
| 0509040ea3fe50071181bbc359eb7738 | 87 |
| 056b4ada5bbc2c50cc7842547dda6b51 | 64 |

**These are the sellers who have fulfilled more orders than the average seller on the platform.**

# 12 Highest-Rated States by Average Review Score

```sql
SELECT c.customer_state,
AVG(review_score) AS avg_review
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
JOIN order_reviews r
ON o.order_id = r.order_id
WHERE o.order_status = "delivered"
GROUP BY c.customer_state
ORDER BY avg_review DESC;
```

**Output**

| customer_state | avg_review |
|---|---|
| SP | 4.2462 |
| AP | 4.2424 |
| PR | 4.2383 |
| AM | 4.2207 |
| MG | 4.1927 |
| RS | 4.1855 |
| MS | 4.1690 |
| RO | 4.1653 |
| TO | 4.1538 |
| RN | 4.1501 |
| MT | 4.1474 |
| DF | 4.1316 |
| SC | 4.1311 |
| GO | 4.1029 |

These states are ranked by customer review scores for delivered orders. For the lower-performing states, the team can work with logistics partners to improve performance.

13

# Customers Who Never Left a Review

```sql
SELECT c.customer_id
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
LEFT JOIN order_reviews r
ON o.order_id = r.order_id
GROUP BY c.customer_id
HAVING COUNT(r.review_score) = 0;
```

**Output**

| customer_id |
|---|
| 0093a68265f18be3adbbe054dc4c9295 |
| 00f83de98e791c2a4b779f7e61a4cf28 |
| 00ffbb23812bc76062322afd86d8cbf7 |
| 010714eb89c36dd92fee08385cc1dd6b |
| 0125ed303758dc1d7e4d99b000f88768 |
| 0145ffd2fcca22ddeebbcd95f7ad890e |
| 021ed85d23b734289c4dd0c827a3559e |
| 0298eec43baa253837fca6e98da66903 |
| 02f0b823e1d7f9b63b25092db3208bdf |
| 030ae7b844c8db8738bb14019351ea55 |
| 034dbf527c403a97312d2d20abb0d0d3 |
| 034ecc550de79becac426a8ed79722f9 |
| 0419432b440cac406896f9b2ed4419a1 |
| 04361b218639fa1eea85ab84aebcab7f |
| 05918175d2c82af715f5ca32bd7c5e9d |

These are the customers the team can target with review reminder emails and in-app nudges to improve engagement

# 14 Month With the Highest Order Volume

amazon

```
WITH a AS (SELECT DATE_FORMAT(order_purchase_timestamp, "%Y-%m")
AS order_month,
COUNT(order_id) AS order_count
FROM orders
GROUP BY order_month)

SELECT order_month, order_count
FROM (SELECT *,
DENSE_RANK() OVER(ORDER BY order_count DESC) AS rnk
FROM a) AS b
WHERE rnk = 1;
```

**Output**

| | order_month | order_count |
|---|---|---|
| ▶ | 2017-11 | 7544 |

**November 2017 was the month with the highest number of orders recorded to date.**

amazon

# Thank You

“Behind every delivery is a data point, and behind every data point is a better decision.”